

Predicate Logic with the consideration of $\perp$ (continue)

- $\perp$ cannot satisfy any predicate: $\perp \not\models p$ for all p , even if p is the constant T .
 - Runtime errors can appear while evaluating a predicate p . In general, we now have **three possible situations for a proper state trying to satisfy a predicate**: $\sigma \models p$ (in other words $\sigma(p) = T$), $\sigma \models \neg p$ (in other words $\sigma(p) = F$), or $\sigma(p) = \perp$.
 - Previously we have $\sigma \not\models p$ means $\sigma \models \neg p$, but this is no longer true when $\perp$ is taken into consideration. $\sigma \not\models p$ means “It is not true that $\sigma \models p$ ” and it means $\sigma \models \neg p$ or $\sigma(p) = \perp$.
- True or False.
 - If $\sigma \models p$, then $\sigma \not\models \neg p$. True.
 - If $\sigma \models p$, then $\sigma(p) \neq \perp$. True.
 - If $\sigma \models \neg p$, then $\sigma \not\models p$. True.
 - If $\sigma \not\models \neg p$, then $\sigma \models p$. False.
 - If $\sigma(p) = \perp$, then $\sigma(\neg p) = \perp$. True.
 - If $\sigma(p) \neq \perp$, then $\sigma \not\models p \Leftrightarrow \sigma \models \neg p$. True.
 - If p is a valid predicate (in other words, $\models p$), then $\perp \models p$. False.
 - If $\not\models p$, then there exists some σ such that $\sigma \not\models p$. True.
 - If $\not\models p$, then there exists some σ such that $\sigma \models \neg p$. False. It means there exists some state σ such that $\sigma(p) = \perp$ or $\sigma(p) = F$.

(Satisfaction by a collection of states)

- We usually use Σ to represent the collection of all (well-formed) states; and we usually use $\Sigma_{\perp} = \Sigma \cup \{\perp\}$.
 - Let $\Sigma_0 \subseteq \Sigma_{\perp}$, we say $\Sigma_0 \models p$, if for every state $\sigma \in \Sigma_0$ we have $\sigma \models p$.
- True or False.
 - Let $\sigma \in \Sigma$, then $\sigma \neq \perp$. True
 - Let $\Sigma_0 \subseteq \Sigma_{\perp}$, then $\perp \in \Sigma_0$. False, $\perp$ is allowed to be in such Σ_0 , but not necessarily.
 - Let $\Sigma_0 \models p$, then $\perp \notin \Sigma_0$. True, $\perp$ doesn't satisfy any predicate.
 - If $\Sigma_0 \not\models p$, then there exists some $\sigma \in \Sigma_0$ such that $\sigma \not\models p$. True
 - $\emptyset \models p$ ($\emptyset$ is an empty set of states) True, “every state” in $\emptyset$ satisfies p
 - $\emptyset \models F$ ($\emptyset$ is an empty set of states) True, it means “no state” satisfies F
 - $\Sigma_0 \models p \wedge \Sigma_0 \models \neg p$ if and only if $\Sigma_0 = \emptyset$ True

For questions h and i, we assume that p cannot be evaluated to $\perp$ in any state.

- Let $\Sigma_0 - \perp = \{\tau\}$, then $\Sigma_0 - \perp \models p$ or $\Sigma_0 - \perp \models \neg p$ True
- $\Sigma_0 - \perp \models p$ or $\Sigma_0 - \perp \models \neg p$ False, it is possible that some states in $\Sigma_0 - \perp$ satisfy p and some satisfy $\neg p$, then $\Sigma_0 - \perp \not\models p$ and $\Sigma_0 - \perp \not\models \neg p$.

Nondeterminism

- Nondeterminism is a theoretical idea, in general it means “don’t make decision, consider all possible outcomes at the same time with same probability.” Note that, it doesn’t mean “randomly pick possible outcome”. Just like Schrodinger’ cat, it is both alive and dead at the same time with a 50% 50% probability.
 - For example, when we say, “choose one side of a coin nondeterministically”, it means “choose head with 50% probability and choose tail with 50% probability.” It doesn’t mean “pick one side randomly” because it will result in either a head or a tail.
 - However, a nondeterministic program can be simulated by “random picking a branch among all possible branches for a lot of times”. For example, you can simulate the procedure “choosing one side of a coin nondeterministically” by tossing a fair coin a lot of times, then you will come up with a set of outputs: {*head*, *tail*}, and the probability of two outputs will be near 50% and 50%.
 - Here is another example. If we have a simple program that returns the maximum between x and y :
if $x \leq y$ then $max := y$ else $max := x$ fi

This **if – else** statement is deterministic. When we have $x = y$, it will always choice y to be the max. Consider a different program that does the following:
“If $x \geq y$ then $max := x$, if $x \leq y$ then $max := y$ ”
and it can evaluate both branches at the same time. When we have $x = y$, then max should have 50% chance to be x and 50% chance to be y .
- Actually, a machine/program cannot run *nondeterministically*, but designing a nondeterministic machine/program is useful:
 - i. A nondeterministic machine/program has a deterministic machine/program that can do the same job (although the deterministic version might need to finish the same job using much longer time). This is a topic in CS530 Computational Theory.
 - ii. The design process can be simplified. We can avoid some insignificant choice-making and only focus on the important decisions in the design. The design of a nondeterministic machine is easier to read and understand.

(Syntax of Nondeterministic Statements in our Programming language)

- A nondeterministic **if – fi** statement: **if $B_1 \rightarrow S_1 \square B_2 \rightarrow S_2 \square \dots \square B_n \rightarrow S_n$ fi**
 - We use box symbols to separate different **arms**.
 - Inside each arm we have a **guarded command** $B_i \rightarrow S_i$ which means “if **guard** B_i is true then it’s okay to run S_i ”.
 - How it works:
 - If none of the guards B_i is true, it aborts with a **runtime error**. This is the fourth primary type of runtime error we see.
 - If exactly one guard B_i is true, then it executes S_i .
 - If more than one guard is true, then **choose one** of the true arms *nondeterministically* and execute the corresponding S_i .
 - Here, “**choose one... nondeterministically**” means, the machine splits into several copies of itself, and each copy follows one of the possible arms in parallel; and each copy holds an equal probability so that the total probability adds up to 1.
- 3. Write a nondeterministic **if – fi** statement that chooses the maximum between x and y .
if $x \geq y \rightarrow max := x \square x \leq y \rightarrow max := y$ fi
- 4. Write a nondeterministic **if – fi** statement to simulate **if B then S_1 else S_2 fi**

if $B \rightarrow S_1 \sqcap \neg B \rightarrow S_2$ **fi**

- We express a nondeterministic **while** loop using a nondeterministic **do – od** statement: **do** $B_1 \rightarrow S_1 \sqcap B_2 \rightarrow S_2 \sqcap \dots \sqcap B_n \rightarrow S_n$ **od**
 - How it works:
 - At the beginning of each iteration, check all guards.
 - If none of the guards B_i is true, then finish the loop.
 - If exactly one B_i is true, then execute S_i and go to the next iteration.
 - If more than one guard is true, then choose one of the true arms *nondeterministically* and execute the corresponding S_i ; then go to the next iteration.

(Denotational Semantics of Nondeterministic Program)

- Because of the nature of nondeterminism, we can end up with more than one state when we finish the execution of a nondeterministic program.
 - The denotational semantics of a deterministic program is one single (possibly pseudo) state: $M(S, \sigma) = \{\tau\}$ where $\langle S, \sigma \rangle \rightarrow^* \langle E, \tau \rangle$ and $\tau \in \Sigma_\perp$.
 - The denotational semantics of a nondeterministic program is a set of (possibly pseudo) states: $M(S, \sigma) = \{\tau \in \Sigma_\perp \mid \langle S, \sigma \rangle \rightarrow^* \langle E, \tau \rangle\}$.
 - Let $IF \equiv \text{if } B_1 \rightarrow S_1 \sqcap B_2 \rightarrow S_2 \sqcap \dots \sqcap B_n \rightarrow S_n \text{ fi}$, $M(IF, \sigma) = \begin{cases} \{\perp_e\}, & \text{if } \sigma(B_1 \vee B_2 \vee \dots \vee B_n) = F \\ \{M(S_i, \sigma) \mid \sigma(B_i) = T\}, & \text{otherwise} \end{cases}$
 - Let $DO \equiv \text{do } B_1 \rightarrow S_1 \sqcap B_2 \rightarrow S_2 \sqcap \dots \sqcap B_n \rightarrow S_n \text{ od}$, $M(DO, \sigma) = \begin{cases} \{\sigma\}, & \text{if } \sigma(B_1 \vee B_2 \vee \dots \vee B_n) = F \\ \{M(DO, M(S_i, \sigma)) \mid \sigma(B_i) = T\}, & \text{otherwise} \end{cases}$
 - Also, in a nondeterministic program we might execute a statement S in a collection of (possibly pseudo) states. Let $\Sigma_0 \subseteq \Sigma_\perp$, then $M(S, \Sigma_0) = \{M(S, \sigma) \mid \sigma \in \Sigma_0\}$. Remind that, we usually use Σ to denote the set of all (well-formed) states. We denote $\Sigma_\perp = \Sigma \cup \{\perp\} = \Sigma \cup \{\perp_d, \perp_e\}$ here.
5. Calculate denotational semantics for each of the following nondeterministic statements and states.
- $S \equiv \text{if } T \rightarrow x := 0 \sqcap T \rightarrow x := 1 \text{ fi}$, σ is an arbitrary state
 Since it is possible to have $\langle S, \sigma \rangle \rightarrow^* \langle E, \sigma[x \mapsto 0] \rangle$ and $\langle S, \sigma \rangle \rightarrow^* \langle E, \sigma[x \mapsto 1] \rangle$, thus $M(S, \sigma) = \{\sigma[x \mapsto 0], \sigma[x \mapsto 1]\}$.
 - $S \equiv \text{if } F \rightarrow x := 0 \sqcap F \rightarrow x := 1 \sqcap T \rightarrow \text{skip} \text{ fi}$, $\sigma = \emptyset$
 There is only one true guard, so $\langle S, \sigma \rangle \rightarrow \langle \text{skip}, \sigma \rangle \rightarrow \langle E, \emptyset \rangle$, thus $M(S, \sigma) = \{\emptyset\}$.
 Remind that, we cannot omit the " $\{\}$ " when there is only one state $\emptyset \in M(S, \sigma)$.
 - $S \equiv \text{if } x \geq y \rightarrow \text{max} := x \sqcap x \leq y \rightarrow \text{max} := y \text{ fi}$, $\sigma = \{x = 1, y = 1\}$
 No matter which arm, we have max gets bind with value 1. Thus $M(S, \sigma) = \{\{x = 1, y = 1, \text{max} = 1\}\}$.
 - $S \equiv \text{do } x + y = 2 \rightarrow x := y/x \sqcap x + y = 2 \rightarrow x := x + 1 \sqcap x + y = 4 \rightarrow y := x \sqcap x + y = 4 \rightarrow x := x - 1; y := y - 1 \text{ od}$, and $\sigma = \{x = 1, y = 3\}$
 - After the first iteration of the **do – od** loop, we can have these states: $\{x = 1, y = 1\}, \{x = 0, y = 2\}$.
 - We execute the second loop in both above states. After the second iteration of the **do – od** loop, we can have these states: $\{x = 1, y = 1\}, \{x = 2, y = 1\}, \perp_e$ and $\{x = 1, y = 2\}$. We noticed that state

- $\{x = 1, y = 1\}$ appears again, and it is the only state that can pass some guard in the next iteration; if we keep evaluating S with $\{x = 1, y = 1\}$ the program will diverge.
- Thus, $M(S, \sigma) = \{\{x = 2, y = 1\}, \{x = 1, y = 2\}, \perp_e, \perp_d\}$.
 - From the above examples, we can see that:
 - The denotational semantics of a nondeterministic program can be only one state. Thus, we can say that “if $M(S, \sigma)$ is a set with more than one state, then S is nondeterministic” but its converse is not true.
 - For a nondeterministic program S , $M(S, \sigma)$ can contain pseudo states, and possibly more than one type of pseudo states.
 - We can say $\tau \in M(S, \sigma)$, it means that τ is one of the possible (pseudo) states after evaluating S in σ ; there might be other states. Similarly, we can also say $\{\tau_1, \tau_2\} \subseteq M(S, \sigma)$, it means that τ_1, τ_2 are two of the possible (pseudo) states after evaluating S in σ .